

Taller 2 - Circuitos secuenciales con SystemVerilog y HDL Studio

Sistemas Digitales

Segundo Cuatrimestre 2026

1. Enunciado

Implementar circuitos secuenciales utilizando SystemVerilog y HDL Studio: flip-flops, registros con habilitación, transferencia de datos entre registros, y el sumador de 4 bits del Taller 1 en un camino de datos.

Para la realización de este taller deberá utilizar el paquete inicial del Taller 2 (devcontainer, esqueletos .sv y testbenches) disponible en la página de la materia.

Se deberá enviar al campus la resolución de **todos** los ejercicios: la carpeta **ejercicios/** completa (los Makefile apuntan a ejercicios anteriores). El plazo de entrega se indica en el campus.

2. HDL Studio

2.1. El entorno

El trabajo se realiza dentro del entorno provisto por la materia (VS Code + devcontainer + HDL Studio). No es necesario instalar herramientas por fuera del contenedor.

El entorno opera en dos modos complementarios. Simulación y síntesis:

- En simulación se ejecuta un testbench, se observan entradas y salidas a lo largo de **ciclos de reloj**, y se comprueba el comportamiento del módulo.
- En síntesis se visualiza el circuito inferido. En este taller tienen que aparecer **flip-flops**, no solo compuertas.

2.2. Detalles adicionales

- a) No se deben cambiar los nombres de los archivos, ni sus puertos o señales, ya que de hacerlo dejarían de funcionar los testbenches y ejercicios sucesivos.
- b) Los testbenches son parte de la especificación: no deben modificarse para lograr que una implementación incorrecta sea aceptada.
- c) Cada módulo nuevo debe **instanciar** los módulos de ejercicios anteriores cuando el enunciado lo pida. No reimplementar esa lógica en un solo bloque.
- d) Solo se puede usar la sintaxis vista en clase y la documentada en **sintaxis.typ**. En este taller entra `always_ff @(posedge clk), <=, if / else if, begin / end` y arreglos. No usar `for`, `generate`, `always_comb`, `case`, `enum` ni `always @(posedge clk)`.
- e) `make sim` copia a la carpeta del ejercicio los .sv de ejercicios anteriores (fuente de verdad: el ejercicio de origen). No editar esas copias. Resolver en orden.
- f) Para sintetizar un módulo que instancia otros: después de `make sim`, agregar esos .sv (ya están en la misma carpeta) al circuito de HDL Studio. No agregar el testbench. *Create circuit* sobre un solo archivo no alcanza: Yosys tira `unknown module`. Detalle en **sintaxis.typ**.

g) **make wave** abre la traza en Surfer. En secuencial hay que mirar **clk**, las entradas **antes** del flanco y **q después**.

3. Ejercicios

Completar el esqueleto provisto de los módulos que se enumeran a continuación. Deberán diseñar cada uno de manera modular, incorporando una funcionalidad por módulo.

Cuando un ejercicio reutiliza un módulo anterior, **make sim** copia ese **.sv** a la carpeta. Completar primero el ejercicio de origen y no editar la copia.

3.1. Ejercicio 1: Flip-flop D

En la carpeta **ejercicios/ej1**, implementar un flip-flop D de 1 bit con reset síncronico activo en alto. En cada flanco ascendente de **clk**:

- si **rst = 1**, **q** pasa a 0;
- si no, **q** pasa a valer **d**.

Entre flancos, **q** conserva su valor aunque **d** cambie.

```
module ff_d (  
    input logic clk,  
    input logic rst,  
    input logic d,  
    output logic q  
);  
    // completar: always_ff @(posedge clk)  
endmodule
```

- Escribir el **always_ff**. Tip: usar **<=**, no **=**.
- Completar a mano, **antes** de simular, **q** luego de cada flanco. Un guion significa “sigue igual”:

Flanco	rst	d	q luego
1	1	1	
2	0	1	
3	0	0	
4	0	1	

- Ejecutar **make sim**. Y sintetizar. Probar el circuito que se genera.
- Abrir **make wave** y observar el comportamiento del circuito.
- ¿**clk** y **rst** son dato o control? ¿Y **d**?

3.2. Ejercicio 2: Registro de 1 bit con enable

En la carpeta **ejercicios/ej2**, implementar un registro de 1 bit con escritura habilitada. **make sim** copia **ff_d.sv** del ejercicio 1.

Prioridad en cada flanco:

- a) si **rst = 1**, **q** pasa a 0;
- b) si no, si **we = 1**, **q** pasa a **din**;
- c) si no, **q** se mantiene en su valor anterior.

```
module registro_1b (  
    input logic clk,  
    input logic rst,  
    input logic we,  
    input logic din,  
    output logic q  
);  
    // instanciar ff_d y un mux  
endmodule
```

- Completar el módulo.
- Ejecutar `make sim`. Y sintetizar. Probar el circuito que se genera.
- Abrir `make wave` y observar el comportamiento del circuito.

3.3. Ejercicio 3: Registro de 4 bits

En la carpeta `ejercicios/ej3`, implementar un registro de 4 bits usando cuatro `registro_1b`. Recordar que `make sim` copia `registro_1b.sv` y `ff_d.sv`.

```
module registro_4b (
    input logic clk,
    input logic rst,
    input logic we,
    input logic [3:0] din,
    output logic [3:0] q
);
    // usar cuatro instancias de registro_1b
endmodule
```

- Completar el módulo.
- Ejecutar `make sim`.
- Y sintetizar. Probar el circuito que se genera.
- `clk` y `rst` se replican a los 4 bits. ¿Siguen siendo control?

3.4. Ejercicio 4: Transferencia entre registros

En la carpeta `ejercicios/ej4`, armar una máquina con 4 registros de 4 bits `R0`, `R1`, `R2` y `R3` que comparten un **bus**. Un multiplexor elige quién habla; los `we` eligen quién escucha. `make sim` copia `registro_4b.sv` (y sus dependencias).

```
module transferencia (
    input logic clk,
    input logic rst,
    input logic [3:0] force_in,
    input logic force_en,
    input logic [1:0] src,
    input logic we0,
    input logic we1,
    input logic we2,
    input logic we3,
    output logic [3:0] r0,
    output logic [3:0] r1,
    output logic [3:0] r2,
    output logic [3:0] r3
);
    // bus y cuatro registro_4b
endmodule
```

Comportamiento:

- si `force_en = 1`, el bus vale `force_in` (sirve para inyectar un valor);
- si no, `src[1:0]` elige: `00` → `r0`, `01` → `r1`, `10` → `r2`, `11` → `r3`.

Los cuatro registros tienen `din = bus`. `we0` escribe `R0`, `we1` escribe `R1`, etc.

Ciclo	force_en	force_in	we0	we1	we2
1					

- Completar el módulo. Ejecutar `make sim`. Y sintetizar. Probar el circuito que se genera.
- Completar la secuencia: poner `4'b1010` en `R0`, copiar `R0` → `R1`, poner `4'b0100` en `R2`, copiar `R2` → `R0`, copiar `R1` → `R2`. El testbench corre exactamente esa receta.

3.5. Ejercicio 5: El sumador de T1 en el camino

En la carpeta ejercicios/ej5, conectar el `sumador_4b` del Taller 1 a tres registros: `R_a`, `R_b` y `R_s`. `make sim` copia los registros anteriores y los `.sv` del sumador (viven en `ejercicios/lib/`; no modificarlos).

```
module datapath_sumador (
    input  logic      clk,
    input  logic      rst,
    input  logic [3:0] force_in,
    input  logic      we_a,
    input  logic      we_b,
    input  logic      we_s,
    output logic [3:0] r_a,
    output logic [3:0] r_b,
    output logic [3:0] r_s,
    output logic      cout
);
    // tres registro_4b + sumador_4b (cin = 0)
endmodule
```

Contrato:

- `Ra` y `Rb` se cargan con `force_in`. Serán los sumandos del módulo.
- `Rs` se carga con la suma de `Ra` y `Rb`. Será el resultado de la suma.
- El sumador **no** tiene reloj: calcula siempre. El `we_s` decide cuándo **guardar**.
- `cin` del sumador va atado a 0. El `cout` no será utilizado en este taller.

No usar el operador `+`. Instanciar `sumador_4b`.

- a) Completar el módulo y sintetizar. Probar el circuito que se genera.
- b) Clasificar `force_in`, `we_a`, `we_b`, `we_s`, `clk`, `r_s`: dato o control.
- c) Completar: cargar `4'b0011` en `R_a` y `4'b0101` en `R_b` (¿cuántos ciclos, con un solo `force_in`?) y después pulsar `we_s`. ¿Cuánto vale `R_s`? ¿`R_a` y `R_b` se mueven?
- d) ¿Por qué `we_a`, `we_b` y `we_s` en el **mismo** flanco no pueden cargar 3 y 5 y guardar 8 a la vez?

Importante: Todos los ejercicios son de entrega obligatoria.